Verification Code Lifecycle

This document explains how a product verification code is created, protected, activated, and checked by goVerifEye.

1. Platform configuration

A platform administrator controls the length of newly generated verification codes through the database-backed option:

`codes.verification_code_length`

The default is **16 digits**. The permitted range is **6 to 28 digits**. This is a global setting, cannot be public or disabled, and changes only affect new batches. Codes already issued continue to work at their original length.

2. Batch creation

An authenticated vendor requests a code batch for one of its active products. The request must include an idempotency key, which prevents a network retry from creating a second batch.

The API checks that:

- the caller belongs to the organization;
- the organization and product are active;
- the requested quantity is between 100 and 10,000 and within the configured server limit;
- product dates are sensible; and
- the same idempotency key has not already completed a batch.

The batch, its codes, and the product counter are written in one database transaction. If any part fails, the whole operation is rolled back.

3. Secure generation

For each label, the backend creates two separate numeric values using Node.js cryptographic random generation:

Value	Purpose	Stored?
Verification code	Public code printed on the product label and entered by a customer	Yes
Activation code	Private code used by the vendor to activate the label	Only as a protected hash

The generator rejects codes that look too obvious:

- more than two leading zeroes;
- all digits the same, such as `111111111111`;
- four or more repeated digits in a row;

- ascending or descending digit runs of six or more, such as `123456` or `987654`.

The database has a unique constraint on verification codes. The service also checks a new batch for duplicates before saving it.

4. Protecting activation codes

The activation code is never stored as plain text. Instead, the backend saves an HMAC-SHA256 value:

```
HMAC key: CODE_ACTIVATION_PEPPER (server environment secret)
HMAC message: verificationCode:activationCode
```

The activation code is returned only in the initial batch-generation response. A replayed idempotent request deliberately returns no credentials. The vendor must therefore store the original generated file securely.

5. Activation

To activate a label, an authenticated vendor submits its verification code and activation code. The API locks the code record while it checks it, then recreates the HMAC and compares the two hashes using a timing-safe comparison.

On success, the code becomes `active` and records who activated it and when. On failure, the attempt count increases. After five failed attempts, the code is suspended. The activation endpoint is also rate-limited.

6. Customer verification

Customers submit only the printed verification code. The public endpoint is rate-limited and accepts numeric codes from 6 to 28 digits, allowing old and newly configured code lengths to coexist.

The API confirms that the code exists, is active, and belongs to an active product. It then returns the product-verification result, increments the scan count, and records a verification event.

7. Fraud and audit trail

Every successful verification records the time, supplied location/complaint, IP address, and user agent hash for risk analysis. Repeated scans and unusually frequent checks can be marked suspicious for review.

Batch creation, activation, suspension, and related protected actions flow through the authenticated API and audit logging. This provides accountability without storing activation secrets in logs or the database.

8. Web and mobile offline checks

The web and mobile apps may perform a quick offline check before calling the API: the value must contain digits only and be **6 to 28 digits** long. This avoids unnecessary requests, but it does not verify that a code is real, active, or assigned to a product; only the backend can do that.

An administrator may change the current generation length anywhere within the 6–28 digit range without rebuilding either app. Both current and previously issued lengths should remain accepted so older labels continue to work.

If the agreed product requirement ever changes the supported range itself (for example, allowing fewer than 6 or more than 28 digits), the mobile app's offline validation must be updated, rebuilt, tested, and released through Google Play and the Apple App Store. Store review and release timing can delay availability. The web app can be redeployed more quickly, but should still be updated and tested with the same policy.

9. OCR scan-to-fill design

OCR is a convenience feature, not offline proof of authenticity. It extracts a likely code from a product image; the backend remains responsible for deciding whether that code is real and active.

1. Scan barcode or QR code first

If the label contains a barcode or QR code that embeds the verification code, the app should use it before OCR. Barcode/QR scanning is faster and more accurate than reading printed text.

2. Guide the camera to the correct area

The scan screen should show a rectangular overlay labelled **"Place the verification number here."** The user captures only this small part of the package, reducing irrelevant packaging text sent into OCR.

3. Run OCR on the cropped image

For mobile, run text recognition on the cropped image on the device. Before recognition, the app should:

- crop to the camera guide;
- correct rotation and perspective where possible;
- convert to grayscale, increase contrast, and enlarge the crop; and
- reject blurred or very dark captures and ask the user to retake the photo.

4. Extract likely codes from noisy text

The app should inspect all OCR text blocks and normalize only likely numeric candidates:

- remove spaces and punctuation;
- correct common OCR mistakes in a numeric candidate only: `O` to `0`, `I`/`l` to `1`, and `S` to `5`;
- keep numeric strings between 6 and 28 digits; and
- score candidates using OCR confidence, proximity to the camera-guide centre, and closeness to the currently configured generation length.

The current configured length must not reject a different valid legacy length. For example, a 16-digit label can remain valid after new code batches start using 20 digits.

5. Ask the user to confirm

When there is one high-confidence candidate, prefill it for confirmation:

```
We found: 482917305816
[Verify code] [Edit]
```

When several candidates are plausible, show the best two or three choices and let the user select one. The app must not silently submit a low-confidence OCR result.

6. Use the normal verification API

Only after user confirmation should the app call the existing verification endpoint. When offline, it should say **"Code captured — connect to the internet to verify"** rather than claiming the product is genuine.

7. Privacy and security

OCR should run on-device by default and discard the image immediately after extraction. Product images must not be uploaded or logged during normal verification. Upload is allowed only when a user explicitly submits an image as fraud evidence. Images should be size-limited and stripped of metadata before any approved upload.

The existing web frontend's hard-coded 16-digit validation must be replaced with a 6–28 digit candidate filter. The current administrator-configured length is used to rank and describe candidates, not to reject valid older labels.

Endpoint	Access	Purpose
`GET /options/:namespace`	Public	Lists active, public global options in a namespace.
`GET /platform/options`	`super_admin`	Lists managed configuration options.
`POST /platform/options`	`super_admin`	Creates a configuration option.
`PATCH /platform/options/:id`	`super_admin`	Updates an option. A `reason` is required.
`GET /platform/options/:id/history`	`super_admin`	Returns the audit history for an option.